rv64ilp32 - The future of 32-bit Linux

Guo Ren <guoren@kernel.org>

https://lwn.net/Articles/838807/

32-bit is waiting for its death.

The future of 32-bit Linux

December 4, 2020
This article was contributed by Arnd Bergmann

The news for processors and system-on-chip (SoC) products these days is all about 64-bit cores powering the latest computers and smartphones, so it's easy to be misled into thinking that all 32-bit technology is obsolete. That quickly leads to the idea of removing support for 32-bit hardware, which would clearly make life easier for kernel developers in a number of ways. At the same time, a majority of embedded systems shipped today do use 32-bit processors, so a valid question is if this will ever change, or if 32-bit will continue to be the best choice for devices that do not require significant resources.

To find an answer, it is worth taking a look at different types of systems supported in Linux today, how they have evolved over time with the introduction of 64-bit processors, why they remain popular, and what challenges these face today and in the future.

32-Bit desktops

Linux was first written as a desktop system for IBM PC compatibles, and was eventually ported to almost every other desktop platform available in the 1990s, including a lot of the early Unix workstations across all architectures. Over time, all of these were replaced with 64-bit machines or they disappeared from the market, or in case of Unix workstations, both. The earliest i386, Arm, MIPS, and PowerPC processors all got phased out over time, but a lot of others still remain.

The table shows the 32-bit desktop platforms that proved popular enough to make it into mainline Linux and stay there, supported mainly by loyal hobbyists:

Platform	Architectures	Earliest supported machine	Last supported 32-bit machine	End of marketing	Replaced by
Unix workstation	m68k MIPS SPARC PA-RISC PowerPC	Sun-3/60 (1985)	IBM RS/6000 43P-240, HP Visualize B180L (1997)	1999	x86 Linux, 64-bit Unix
Apple Macintosh	m68k PowerPC ia32	Mac II (1987)	Macbook (2006)	2007	x86-64, arm64 Mac

32ilp32 v.s. 64ilp32 v.s. 64lp64

	32ilp32 (Traditional 32-bit ABI)	64ilp32 (New 32-bit ABI)	64lp64 (Traditional 64-bit ABI)
pointer	**32**	**32**	**64**
ISA	**32**	**64**	**64**

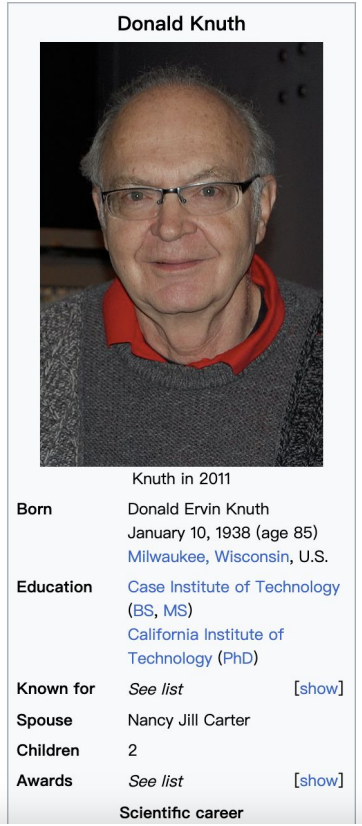
- **Why 32-bit pointer?**
- **Why 64-bit ISA?**

1. Why 32-bit pointer?

2. Why 64-bit ISA?

It is **absolutely idiotic** to have 64-bit pointers when I compile a program that uses less than 4 gigabytes of RAM. When such pointer values appear inside a struct, they not only waste half the memory, they effectively throw away half of the cache. – Knuth (2008)

https://www-cs-faculty.stanford.edu/~knuth/news08.html
A Flame About 64-bit Pointers



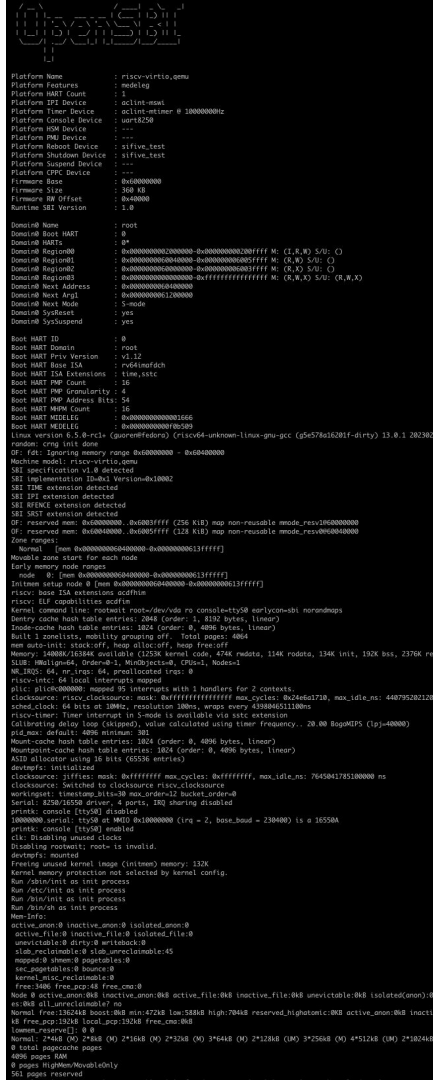
LP64 waste **25%** memory

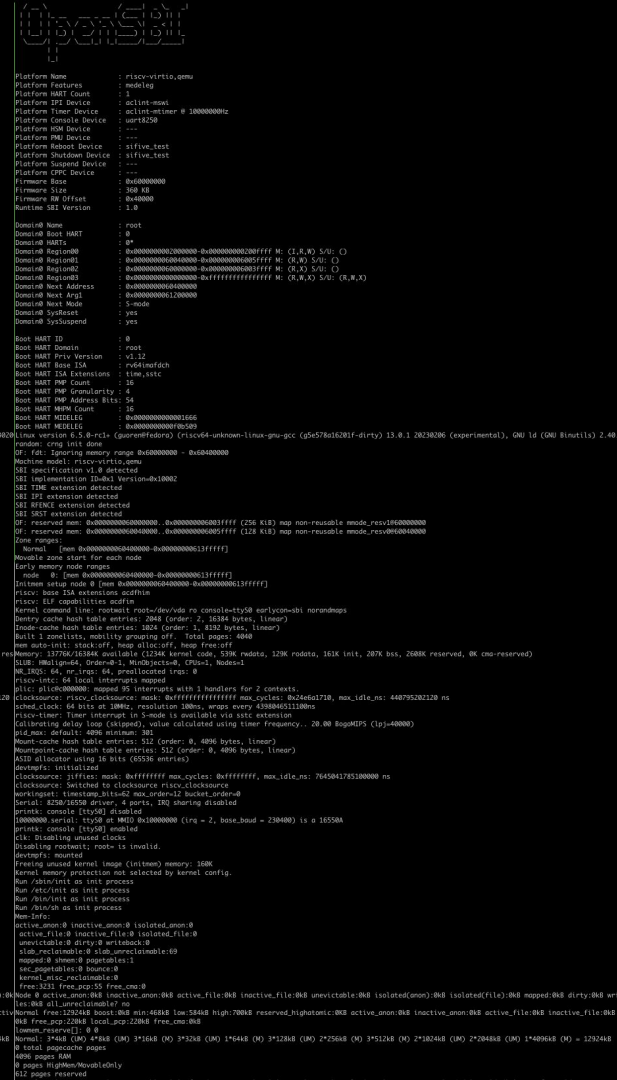
Test Environment:

- Total mem is 16MB (4096 Pages)
- Using tinyconfig
- The dmesg is line-by-line aligned.

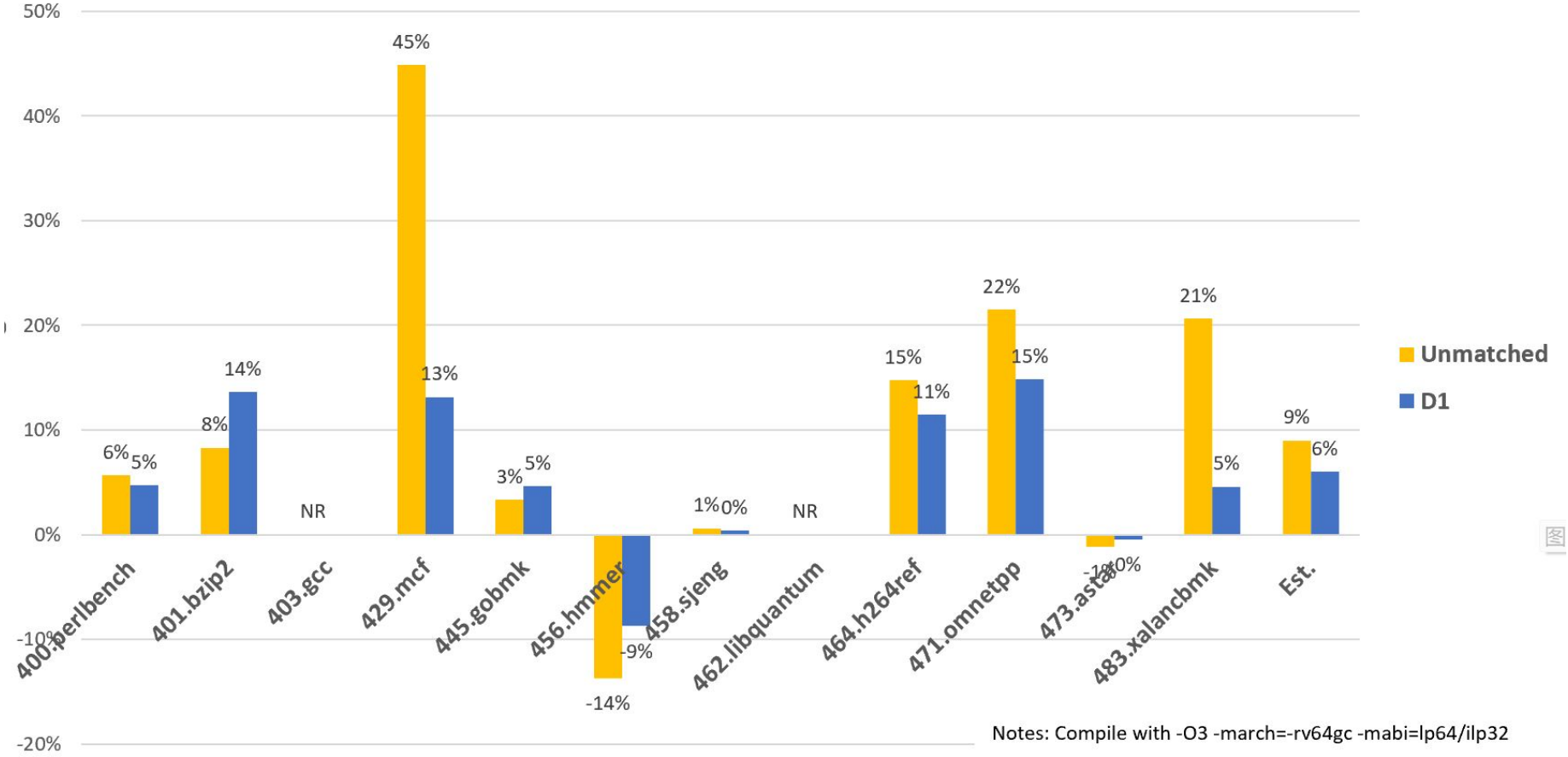
ilp32 = (4096 - 3406) = 690 pages
lp64 = (4096 - 3231) = 865 pages
(865 - 690)/690 = 25%

sizeof(xxx)	ILP32	LP64
struct page	32	64
list_head	8	16
hlist_node	8	16
vm_area_struct …	68	136

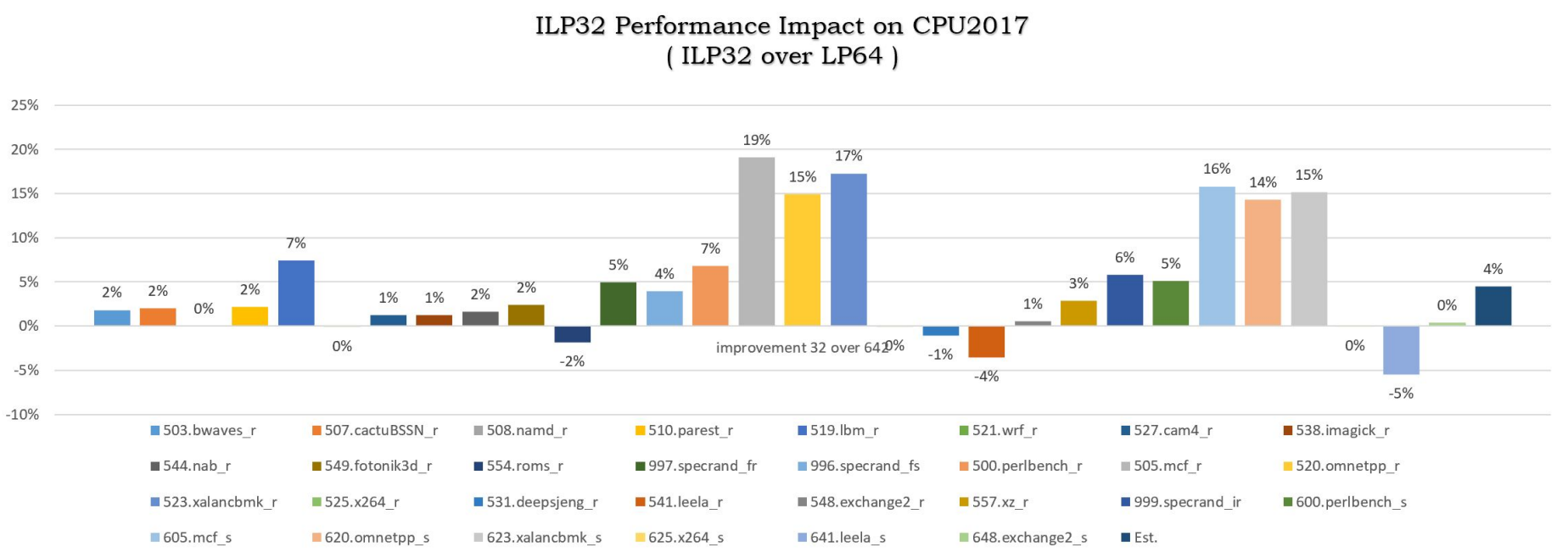




ILP32 Performance Impact on CPU2006 INTSPEED
(ILP32 over LP64)



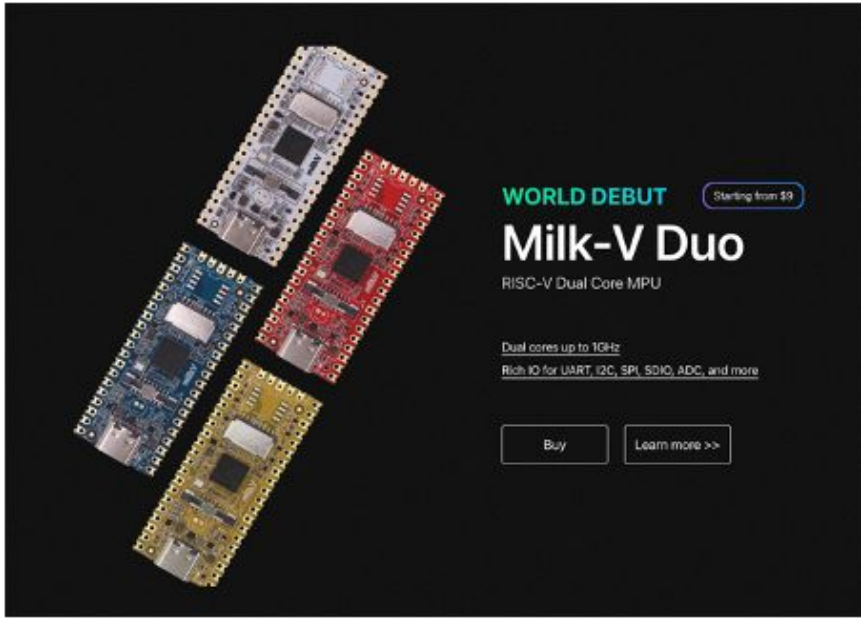
Notes: Compile with -O3 -march=-rv64gc -mabi=lp64/ilp32


ILP32 Performance Impact on CPU2017
(ILP32 over LP64)
25%
20%
15%
10%
5%
0%
-5%
-10%
2%
2%
0%
2%
7%
0%
1%
1%
2%
2%
-2%
5%
4%
7%
19%
15%
17%
improvement 32 over 64
0%
-1%
-4%
1%
3%
6%
5%
16%
14%
15%
0%
-5%
0%
4%
503.bwaves_r
507.cactuBSSN_r
508.namd_r
510.parest_r
519.lbm_r
521.wrf_r
527.cam4_r
538.imagick_r
544.nab_r
549.fotonik3d_r
554.roms_r
997.specrand_fr
996.specrand_fs
500.perlbench_r
505.mcf_r
520.omnetpp_r
523.xalancbmk_r
525.x264_r
531.deepsjeng_r
541.leela_r
548.exchange2_r
557.xz_r
999.specrand_ir
600.perlbench_s
605.mcf_s
620.omnetpp_s
623.xalancbmk_s
625.x264_s
641.leela_s
648.exchange2_s
Est.

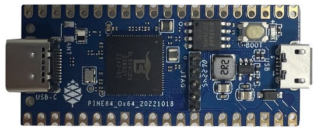
Unmatched

WHY?

Why did they choose 64-bit ISA?







128Mb Ox64 SBC

Community price: $8.00

CORE	• 64-bit 480MHz RV64 C906 • 32-bit 320MHz RV32 E907 • 32-bit 150MHz RV32 E902
MEMORY	• 728KB internal SRAM • 64MB internal PSRAM • 128Mb (16MB) XSPI Flash • microSD slot

1. Why 32-bit pointer?

2. Why 64-bit ISA?

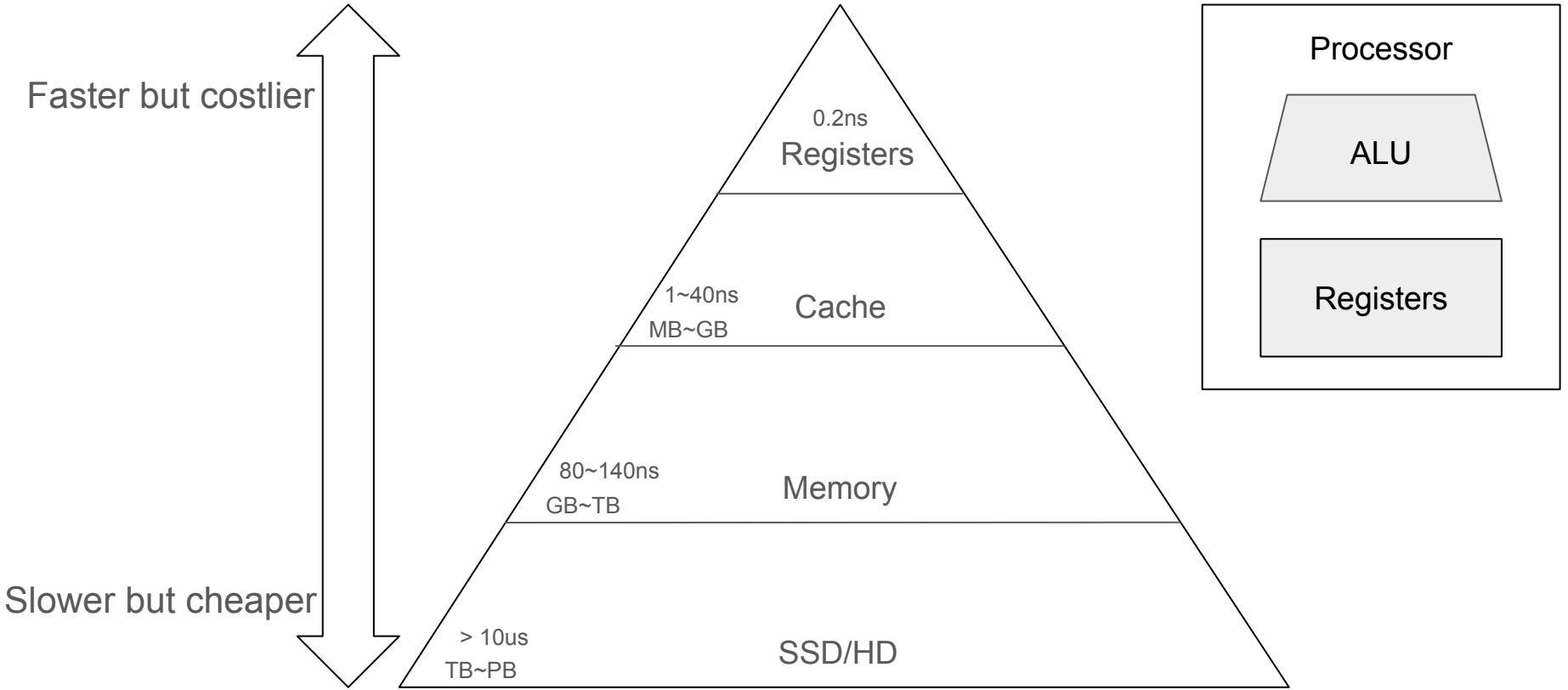
32-bit ISA of Application Processor !?





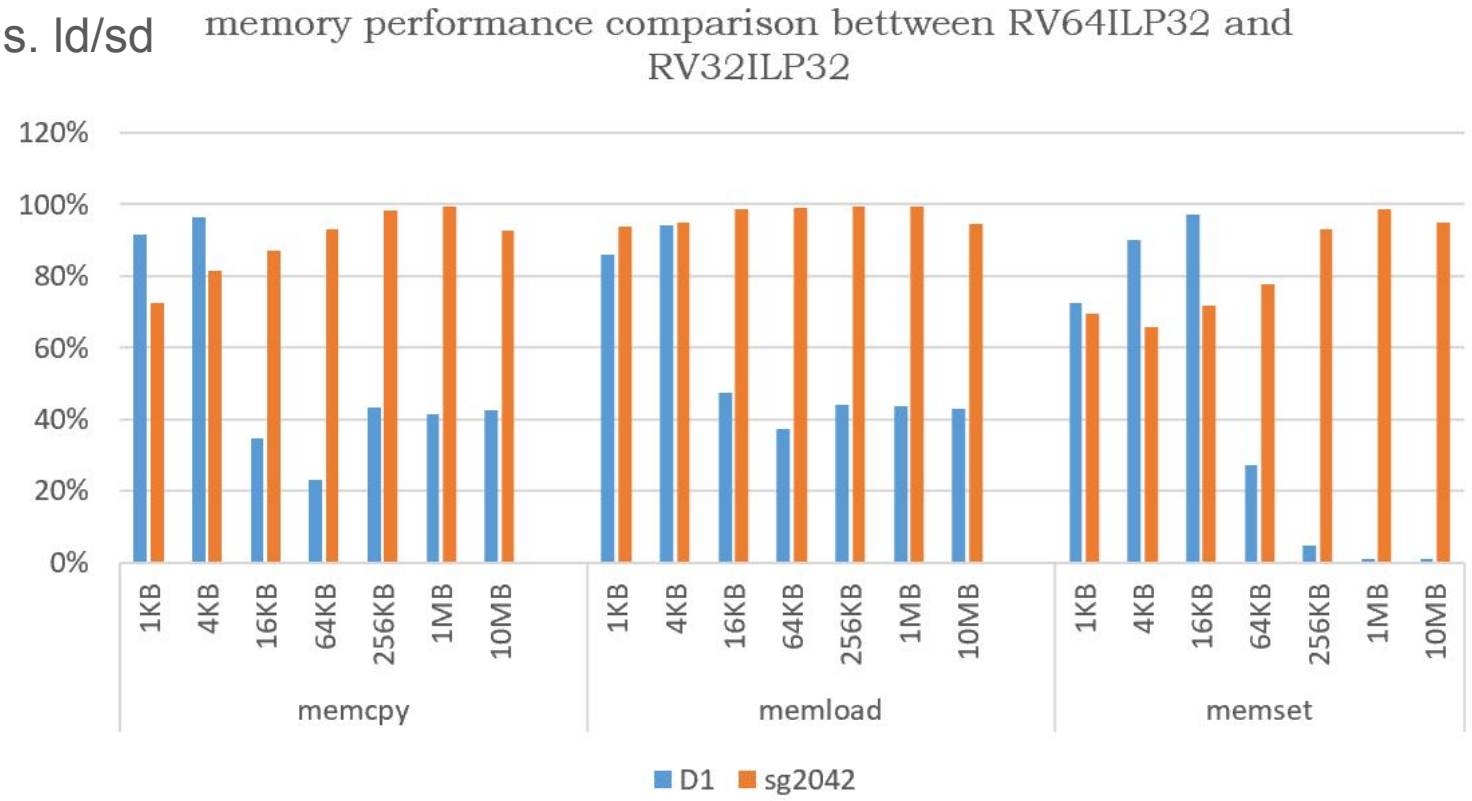
- X86-S killed 32-bit ISA.
- Armv9 deprecated 32-bit ISA.
- RISC-V profiles (RVA23/RVB23/RVA22/RVA20) never include 32-bit ISA.

32-bit mode - Throw away half of the Register



memcpy/memload/memset performance in Linux kernel

lw/sw v.s. ld/sd



Does a pure 32-bit ISA make the chip area smaller?

ARM said: “Compared to Cortex-A35, the Cortex-A32 offers same 32-bit performance but consumes 10% less power and has a 13% smaller core.”
https://community.arm.com/arm-community-blogs/b/architectures-and-processors-blog/posts/introducing-cortex-a32-arm-s-smallest-lowest-power-armv8-a-processor-for-next-generation-32-bit-embedded-applications

1. AArch64 has 31 GPRs, but AArch32 only has 15 GPRs (**16 GPRs removed!**).
2. AArch64 has 32 128-bit SIMD Regs, but AArch32 only has 16 (**16 SIMD Regs removed!**).
3. Cortex-A35 is “**AArch32 + AArch64**”, but Cortex-A32 is only “**AArch32**”.

   13% / 4 = 3.25%

For RISC-V:

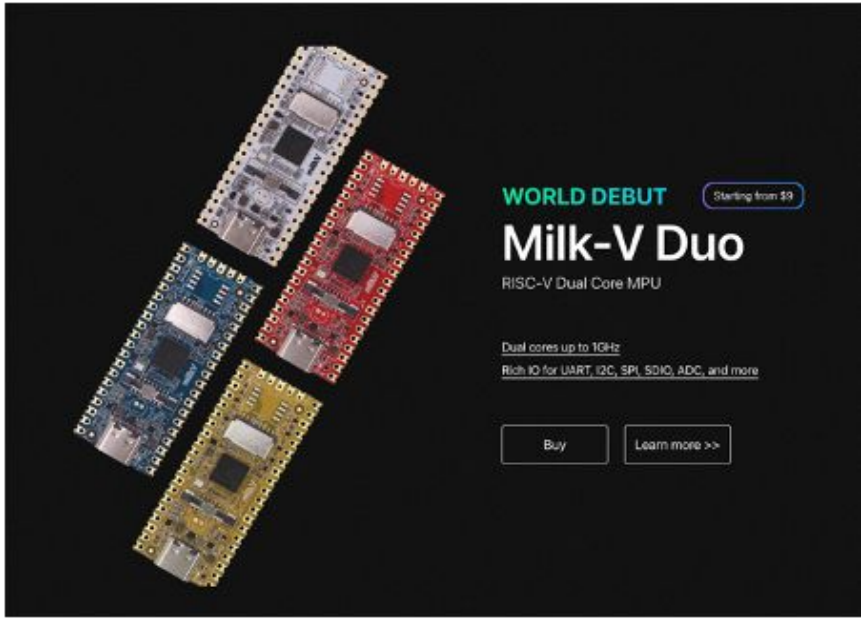
- The number of GPRs, FPU, and VECTOR registers are the same for RV32 and RV64.
- Generally speaking, in embedded application processor scenarios, the CPU core area accounts for no more than 10% of the entire SoC.

  3.25% * 10% = 0.325%

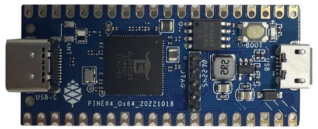
  Therefore, the benefit is too tiny to care.

WISE!

64-bit ISA is a visionary and wise choice!







128Mb Ox64 SBC

Community price: $8.00

CORE	• 64-bit 480MHz RV64 C906 • 32-bit 320MHz RV32 E907 • 32-bit 150MHz RV32 E902
MEMORY	• 728KB internal SRAM • 64MB internal PSRAM • 128Mb (16MB) XSPI Flash • microSD slot

Our Solution: **rv64ilp32**
Run 32-bit pointer on RISC-V 64-bit ISA

The world’s first 64ilp32 ABI Linux kernel!

```
[RFC PATCH V2 00/38] rv64ilp32: Running ILP32 on RV64 ISA
 2023-11-12  6:15 UTC  (39+ messages)
` [RFC PATCH V2 01/38] riscv: u64ilp32: Unify vdso32 & compat_vdso into vdso/Makefile
` [RFC PATCH V2 02/38] riscv: u64ilp32: Remove compat_vdso/
` [RFC PATCH V2 03/38] riscv: u64ilp32: Add time-related vDSO common flow for vdso32
` [RFC PATCH V2 04/38] riscv: u64ilp32: Introduce ILP32 vdso for UXL=64
` [RFC PATCH V2 05/38] riscv: u64ilp32: Adjust vDSO kernel flow for 64ilp32 abi
` [RFC PATCH V2 06/38] riscv: u64ilp32: Add signal support for compat
` [RFC PATCH V2 07/38] riscv: u64ilp32: Add ptrace interface support
` [RFC PATCH V2 08/38] riscv: u64ilp32: Adjust vDSO alternative for 64ilp32 abi
` [RFC PATCH V2 09/38] riscv: u64ilp32: Add xlen_t in user_regs_struct
` [RFC PATCH V2 10/38] riscv: u64ilp32: Remove the restriction of UXL=32
` [RFC PATCH V2 11/38] riscv: u64ilp32: Enable user space runtime switch
` [RFC PATCH V2 12/38] riscv: s64ilp32: Unify ULL & UL into UXL in csr
` [RFC PATCH V2 13/38] riscv: s64ilp32: Introduce xlen_t for 64ILP32 kernel
` [RFC PATCH V2 14/38] riscv: s64ilp32: Add sbi support
` [RFC PATCH V2 15/38] riscv: s64ilp32: Add asid support
` [RFC PATCH V2 16/38] riscv: s64ilp32: Introduce PTR_L and PTR_S
` [RFC PATCH V2 17/38] riscv: s64ilp32: Adjust TASK_SIZE for s64ilp32 kernel
` [RFC PATCH V2 18/38] riscv: s64ilp32: Add ebpf jit support
` [RFC PATCH V2 19/38] riscv: s64ilp32: Add ELF32 support
` [RFC PATCH V2 20/38] riscv: s64ilp32: Add ARCH_RV64ILP32 Kconfig option
` [RFC PATCH V2 21/38] riscv: s64ilp32: Add MMU_SV32 mode support
` [RFC PATCH V2 22/38] riscv: s64ilp32: Add MMU_SV39 "
` [RFC PATCH V2 23/38] riscv: s64ilp32: Enable native atomic64
` [RFC PATCH V2 24/38] riscv: s64ilp32: Add TImode (128 int) support
` [RFC PATCH V2 25/38] riscv: s64ilp32: Implement cmpxchg_double
` [RFC PATCH V2 26/38] riscv: s64ilp32: Disable KVM
` [RFC PATCH V2 27/38] riscv: s64ilp32: Correct the rv64ilp32 stackframe layout
` [RFC PATCH V2 28/38] riscv: s64ilp32: Temporary workaround solution to gcc problem
` [RFC PATCH V2 29/38] riscv: s64ilp32: Introduce ARCH_HAS_64ILP32_KERNEL for syscall
` [RFC PATCH V2 30/38] riscv: s64ilp32: Add u32ilp32 ptrace support
` [RFC PATCH V2 31/38] riscv: s64ilp32: Add u32ilp32 signal support
` [RFC PATCH V2 32/38] riscv: s64ilp32: Validate harts by architecture name
` [RFC PATCH V2 33/38] riscv: s64ilp32: Add rv64ilp32_defconfig
` [RFC PATCH V2 34/38] riscv: Cleanup rv32_defconfig
` [RFC PATCH V2 35/38] clocksource: riscv: s64ilp32: Use __riscv_xlen instead of CONFIG_32BIT
` [RFC PATCH V2 36/38] irqchip: "
` [RFC PATCH V2 37/38] add tinylab defconfig
` [RFC PATCH V2 38/38] 64ilp32 v.s. 64lp64
```

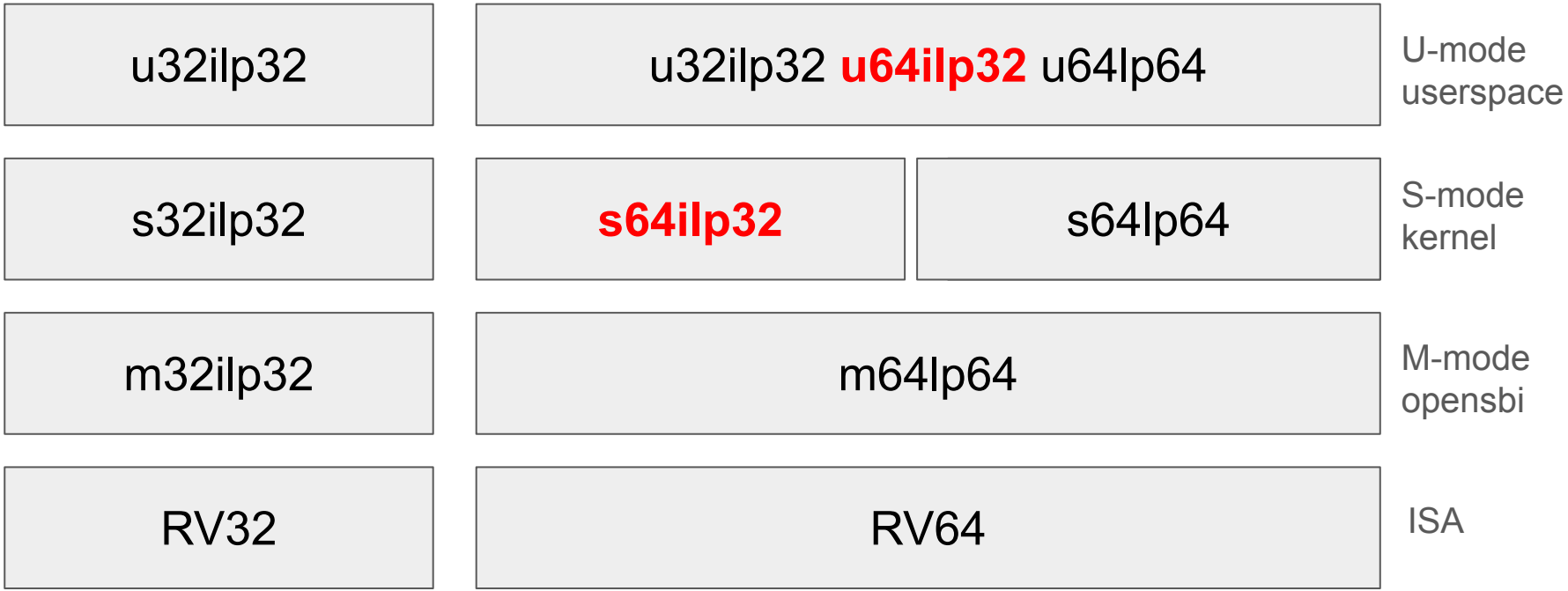
PATCH [01 - 11] u64ilp32
PATCH [12 - 36] s64ilp32

https://lore.kernel.org/linux-riscv/20231112061514.2306187-1-guoren@kernel.org/

u64ilp32: User space support is similar to x86-x32, mips-n32, and arm64-ilp32.

s64ilp32 - The world’s first 64ilp32 ABI Linux kernel!

[RFC PATCH V2] rv64ilp32 patches:



s64ilp32 v.s. s32ilp32

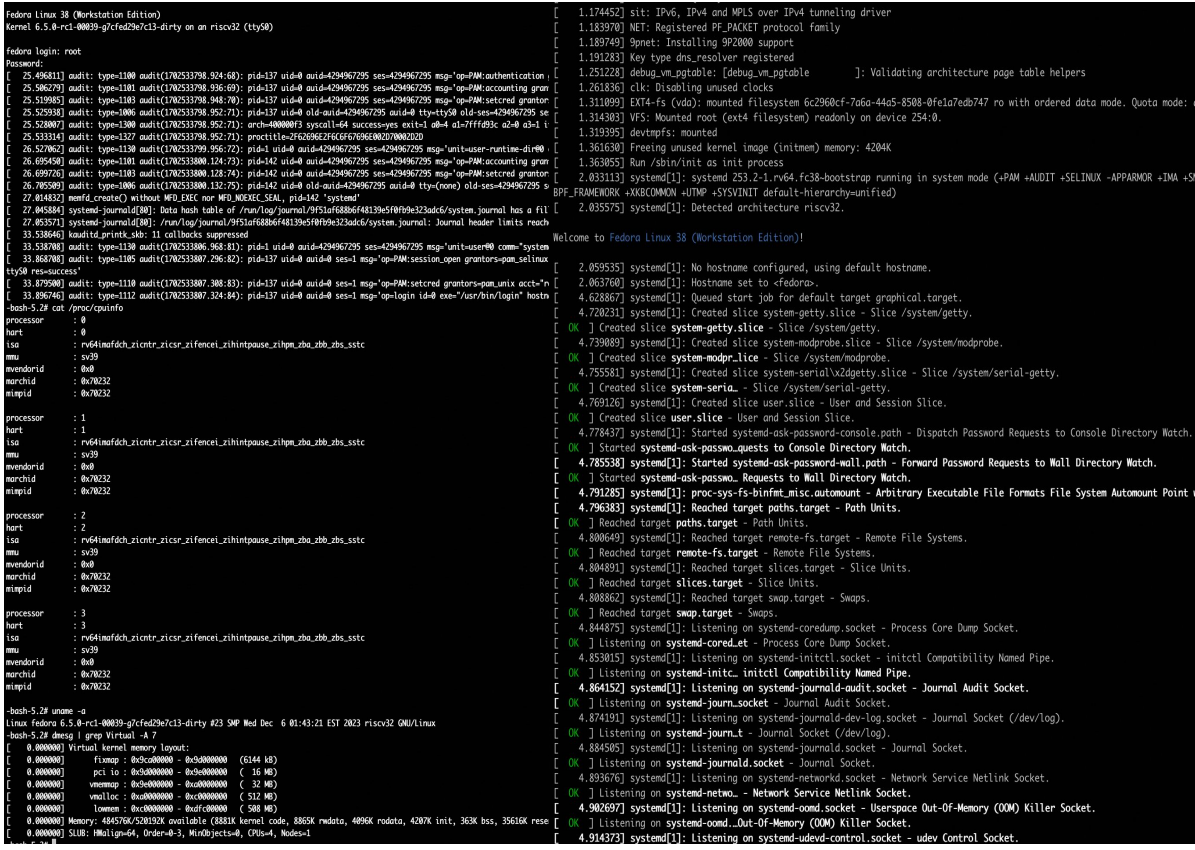
- memcpy/copy_from(to)_user/get(put)_user ld -> lw
- eBPF JIT full -> lite
- Native Atomic64 -> GENERIC_ATOMIC64
- Native 64-bit algorithm ->
  GENERIC_LIB_ASHLDI3
  GENERIC_LIB_ASHRDI3
  GENERIC_LIB_LSHRDI3
  GENERIC_LIB_UCMPDI2
  …

Fedora 38 with “s64ilp32 + u32ilp32”

- Gcc, Binutils, Glibc, Benchmark (PLCT)
- Linux kernel (Alibaba)
- Fedora (PLCT & Redhat)

1800+ rv32 fedora packages

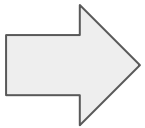
- XuanTie C908 (Kendryte K230)
- XuanTie C907 (Coming soon)



Next: [RFC PATCH V3] s64ilp32 + u64lp64

Proof of Concept:

```
# cat /proc/1/maps
555555555000-555555640000 r-xp 00000000 fe:00 17                         /bin/busybox
555555640000-555555643000 r--p 000ea000 fe:00 17                         /bin/busybox
555555643000-555555644000 rw-p 000ed000 fe:00 17                         /bin/busybox
555555644000-555555645000 rw-p 00000000 00:00 0                          [heap]
555555645000-555555666000 rw-p 00000000 00:00 0                          [heap]
7ffff7e74000-7ffff7e76000 rw-p 00000000 00:00 0
7ffff7e76000-7ffff7fb8000 r-xp 00000000 fe:00 137                        /lib/libc.so.6
7ffff7fb8000-7ffff7fb9000 ---p 00142000 fe:00 137                        /lib/libc.so.6
7ffff7fb9000-7ffff7fbc000 r--p 00142000 fe:00 137                        /lib/libc.so.6
7ffff7fbc000-7ffff7fbe000 rw-p 00145000 fe:00 137                        /lib/libc.so.6
7ffff7fbe000-7ffff7fcb000 rw-p 00000000 00:00 0
7ffff7fcb000-7ffff7fd4000 r-xp 00000000 fe:00 146                        /lib/libresolv.so.2
7ffff7fd4000-7ffff7fd5000 ---p 00009000 fe:00 146                        /lib/libresolv.so.2
7ffff7fd5000-7ffff7fd6000 r--p 00009000 fe:00 146                        /lib/libresolv.so.2
7ffff7fd6000-7ffff7fd7000 rw-p 0000a000 fe:00 146                        /lib/libresolv.so.2
7ffff7fd7000-7ffff7fd9000 rw-p 00000000 00:00 0
7ffff7fd9000-7ffff7fdb000 r--p 00000000 00:00 0                          [vvar]
7ffff7fdb000-7ffff7fdd000 r-xp 00000000 00:00 0                          [vdso]
7ffff7fdd000-7ffff7ffc000 r-xp 00000000 fe:00 132                        /lib/ld-linux-riscv64-lp64d.so.1
7ffff7ffd000-7ffff7ffe000 r--p 0001f000 fe:00 132                        /lib/ld-linux-riscv64-lp64d.so.1
7ffff7ffe000-7ffff8000000 rw-p 00020000 fe:00 132                        /lib/ld-linux-riscv64-lp64d.so.1
7fffffffdf000-800000000000 rw-p 00000000 00:00 0                         [stack]
```

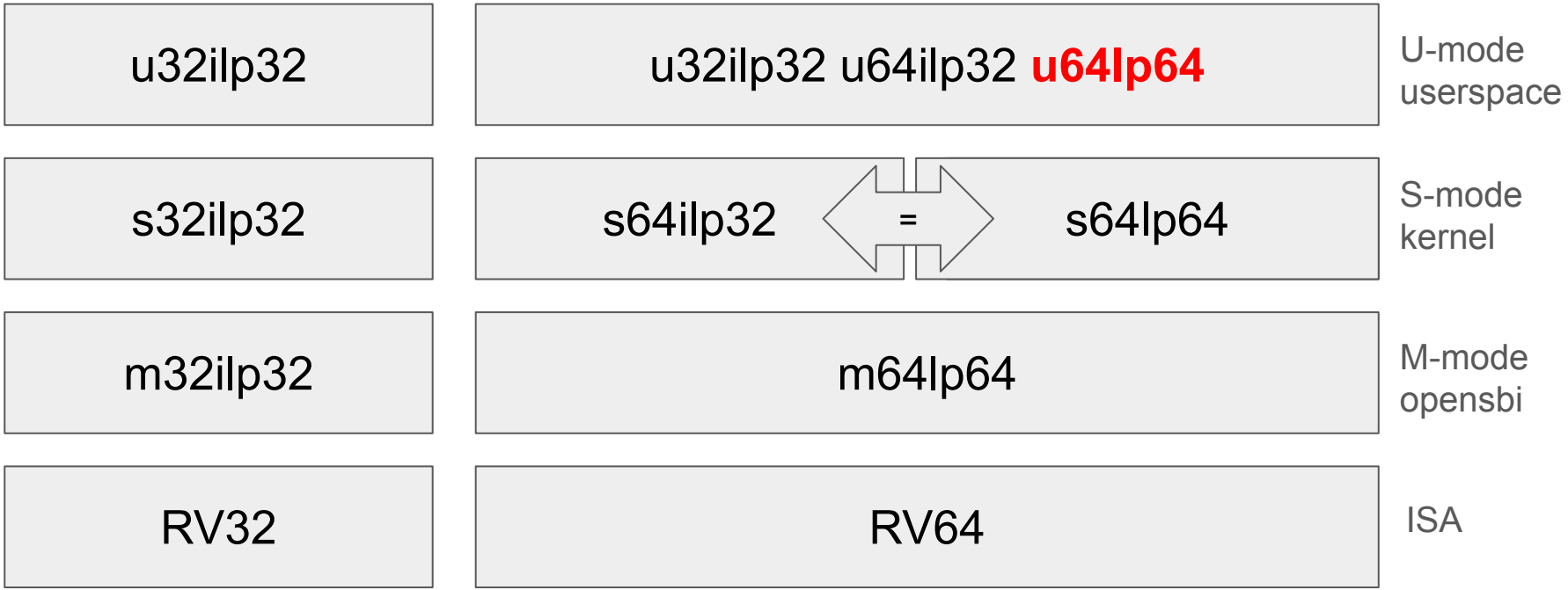


```
# cat /proc/1/maps
55555000-55640000 r-xp 00000000 fe:00 17                         /bin/busybox
55640000-55643000 r--p 000ea000 fe:00 17                         /bin/busybox
55643000-55644000 rw-p 000ed000 fe:00 17                         /bin/busybox
55644000-55645000 rw-p 00000000 00:00 0                          [heap]
55645000-55666000 rw-p 00000000 00:00 0                          [heap]
77e74000-77e76000 rw-p 00000000 00:00 0
77e76000-77fb8000 r-xp 00000000 fe:00 137                        /lib/libc.so.6
77fb8000-77fb9000 ---p 00142000 fe:00 137                        /lib/libc.so.6
77fb9000-77fbc000 r--p 00142000 fe:00 137                        /lib/libc.so.6
77fbc000-77fbe000 rw-p 00145000 fe:00 137                        /lib/libc.so.6
77fbe000-77fcb000 rw-p 00000000 00:00 0
77fcb000-77fd4000 r-xp 00000000 fe:00 146                        /lib/libresolv.so.2
77fd4000-77fd5000 ---p 00009000 fe:00 146                        /lib/libresolv.so.2
77fd5000-77fd6000 r--p 00009000 fe:00 146                        /lib/libresolv.so.2
77fd6000-77fd7000 rw-p 0000a000 fe:00 146                        /lib/libresolv.so.2
77fd7000-77fd9000 rw-p 00000000 00:00 0
77fd9000-77fdb000 r--p 00000000 00:00 0                          [vvar]
77fdb000-77fdd000 r-xp 00000000 00:00 0                          [vdso]
77fdd000-77ffc000 r-xp 00000000 fe:00 132                        /lib/ld-linux-riscv64-lp64d.so.1
77ffd000-77ffe000 r--p 0001f000 fe:00 132                        /lib/ld-linux-riscv64-lp64d.so.1
77ffe000-78000000 rw-p 00020000 fe:00 132                        /lib/ld-linux-riscv64-lp64d.so.1
7ffdf000-80000000 rw-p 00000000 00:00 0                          [stack]
```

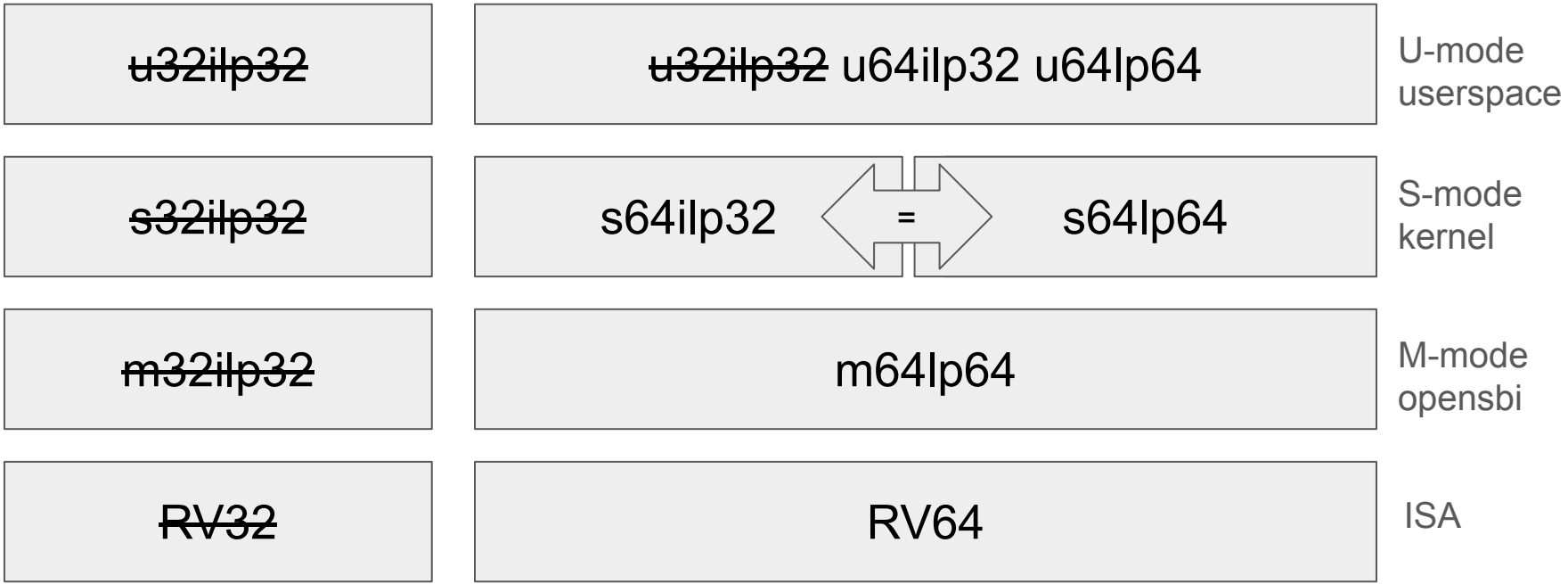
s64lp64 + u64lp64 (128TB)

s64ilp32 + u64lp64 (2GB)

Next: [RFC PATCH V3] s64ilp32 + u64lp64



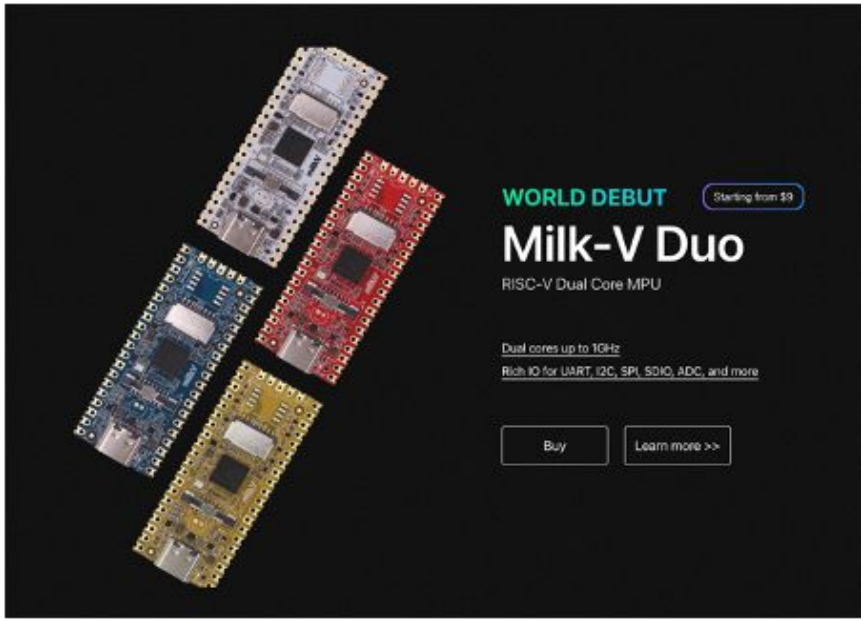
Final Goal: s64ilp32 + u64ilp32



Reuse the 64-bit system call table, then delete the 32-bit ISA and its 32-bit system call table from Linux.

SUCCESS!

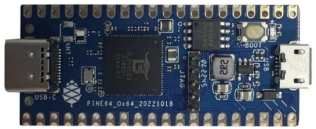
rv64ilp32 ensures these chips succeed!





128Mb Ox64 SBC

Community price: $8.00



CORE	• 64-bit 480MHz RV64 C906 • 32-bit 320MHz RV32 E907 • 32-bit 150MHz RV32 E902
MEMORY	• 728KB internal SRAM • 64MB internal PSRAM • 128Mb (16MB) XSPI Flash • microSD slot

The future of 32-bit Linux - rv64ilp32

END

Backup

Linux doesn’t like 32-bit ISA

eBPF JIT

- The eBPF registers are 64-bit, while ISA registers are 32-bit. BPF registers either map directly to 2 registers, or reside in stack scratch space and are saved and restored when used.
- Many 64-bit ALU operations do not trivially map to 32-bit operations. Operations that move bits between high and low words, such as ADD, LSH, MUL, and others must emulate the 64-bit behavior in terms of 32-bit instructions.

ref: https://lore.kernel.org/netdev/20200220041608.30289-1-lukenels@cs.washington.edu/

Use native 64-bit ALU insns improve crypto algorithms

```
/*
 * On some 32-bit architectures (h8300), GCC ends up using
 * over 1 KB of stack if we inline the round calculation into the loop
 * in keccakf(). On the other hand, on 64-bit architectures with plenty
 * of [64-bit wide] general purpose registers, not inlining it severely
 * hurts performance. So let's use 64-bitness as a heuristic to decide
 * whether to inline or not.
 */
#ifdef CONFIG_64BIT
#define SHA3_INLINE     inline
#else
#define SHA3_INLINE     noinline
#endif
```

```
/* update the state with given number of rounds */

static SHA3_INLINE void keccakf_round(u64 st[25])
{
        u64 t[5], tt, bc[5];

        /* Theta */
        bc[0] = st[0] ^ st[5] ^ st[10] ^ st[15
```

Use native 64-bit load/store improve user space access

```
static int rseq_get_rseq_cs(struct task_struct *t, struct rseq_cs *rseq_cs)
{
…

#if CONFIG_64BIT
        if (get_user(ptr, &t->rseq->rseq_cs))
                return -EFAULT;
#else
        if (copy_from_user(&ptr, &t->rseq->rseq_cs, sizeof(ptr)))
                return -EFAULT;
#endif
```

Use native 64-bit atomic implement CMPXCHG_DOUBLE

mm/slub.c:

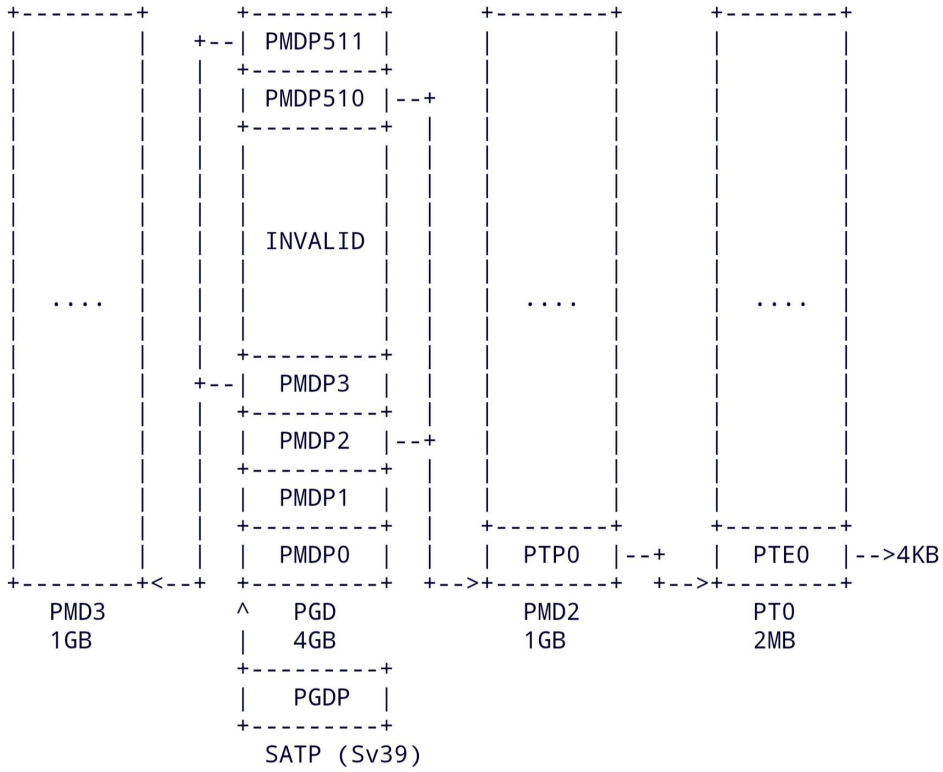
```
if (s->flags & __CMPXCHG_DOUBLE) {
        ret = __update_freelist_fast(slab, freelist_old, counters_old,
                        freelist_new, counters_new);
} else {
        ret = __update_freelist_slow(slab, freelist
```

Improvements

Sign-extend addressing

Traditional x86-x32, mips-n32, and arm64-ilp32 all use zero-extend addressing and the compiler needs to **insert additional zero-extend instructions**, which causes code size and performance problems.

So rv64ilp32 introduces a new solution called **sign-extend addressing**.



Stack size optimization

Traditional x86-x32, mips-n32, and arm64-ilp32 all use 64-bit for callee-saved registers, but they **waste half of the stack size in ILP32** scenarios.

So rv64ilp32 prepares to use **32-bit stack layout** for callee-saved registers.

```
Callee saved the register width
-------------------------------
For 64-bit ISA (including 64lp64, 64ilp32), callee can't determine the
correct width used in the register, so they saved the maximum width of
the ISA register, i.e., xlen size. We also found this rule in x86-x32,
mips-n32, and aarch64ilp32, which comes from 64lp64. See PATCH [20]

Here are two downsides of this:
 - It would cause a difference with 32ilp32's stack frame, and s64ilp32
   reuses 32ilp32 software stack. Thus, many additional compatible
   problems would happen during the porting of 64ilp32 software.
 - It also increases the budget of the stack usage.
   <setup_vm>:
     auipc   a3,0xff3fb
     add     a3,a3,1234 # c0000000
     li      a5,-1
     lui     a4,0xc0000
     addw    sp,sp,-96
     srl     a5,a5,0x20
     subw    a4,a4,a3
     auipc   a2,0x111a
     add     a2,a2,1212 # c1d1f000
     sd      s0,80(sp)----+
     sd      s1,72(sp)    |
     sd      s2,64(sp)    |
     sd      s7,24(sp)    |
     sd      s8,16(sp)    |
     sd      s9,8(sp)     |-> All <= 32b widths, but occupy 64b
     sd      ra,88(sp)    |   stack space.
     sd      s3,56(sp)    |   Affect memory footprint & cache
     sd      s4,48(sp)    |   performance.
     sd      s5,40(sp)    |
     sd      s6,32(sp)    |
     sd      s10,0(sp)----+
     sll     a1,a4,0x20
     subw    a2,a2,a3
     and     a4,a4,a5

So here is a proposal to riscv 64ilp32 ABI:
 - Let the compiler prevent callee saving ">32b variables" in
   callee-registers. (Q: We need to measure, how the influence of
   64b variables cross function call?)
```

Current Problems

y2038 problem

The traditional 32-bit Linux will stop working in 2038 when the 32-bit time_t overflows, which is historical ills.

For rv64ilp32:

- Move to 64-bit system call table

GCC problem

```
64-bit Optimization problem
---------------------------
There is an existing problem in 64ilp32 gcc that combines two pointers
in one register. Liao is solving that problem. Before he finishes the
job, we could prevent it with a simple noinline attribute, fortunately.
    struct path {
            struct vfsmount *mnt;
            struct dentry *dentry;
    } __randomize_layout;

    struct nameidata {
            struct path     path;
            ...
            struct path     root;
    ...
    } __randomize_layout;

            struct nameidata *nd
            ...
            nd->path = nd->root;
    6c88                    ld      a0,24(s1)
                                    ^^ // a0 contains two pointers
    e088                    sd      a0,0(s1)
            mntget(path->mnt);
            // Need "lw a0,0(s1)" or "a0 << 32; a0 >> 32"
    2a6150ef                jal     c01ce946 <mntget> // bug!
```

https://github.com/Liaoshihua/RV64-ILP32/

Pointer arithmetic and Potiner comparison discussions